



TECNOLÓGICO
NACIONAL DE MÉXICO



INSTITUTO TECNOLÓGICO DEL VALLE DE OAXACA

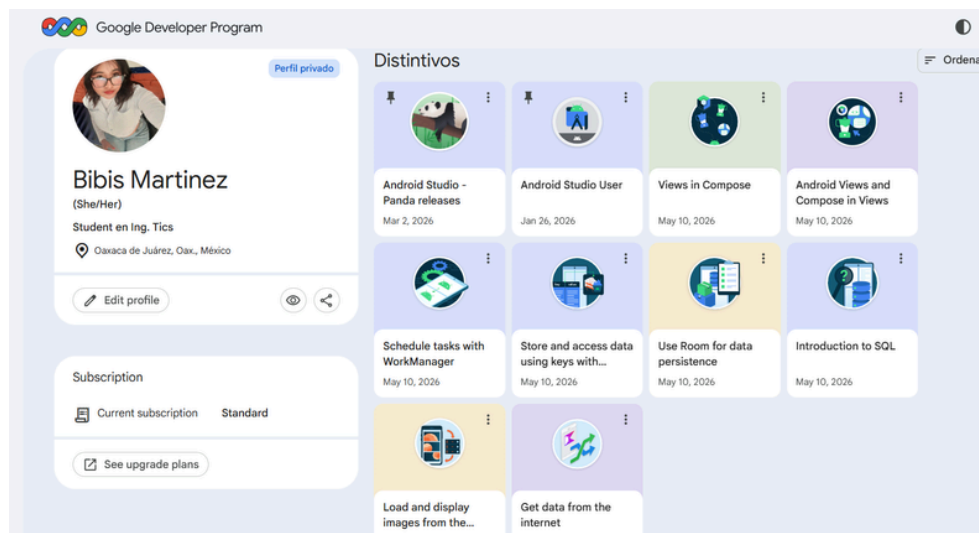
TECNOLOGÍAS DE LA INFORMACIÓN Y LAS
COMUNICACIONES

DESARROLLO DE APLICACIONES MÓVILES

4_1 BITÁCORA SOBRE MANIPULACIÓN DE DATOS (CRUD) (100%)

ALUMNA: ALEYDA BIBIANA MARTINEZ SANCHEZ

PROFESOR: AMBROSIO CARDOSO JIMENEZ



link <https://github.com/bibis03/CRUD-Estadios-Bitacora>

UNIDAD 4

TICS

SEMESTRE 6TO

29/05/2026



4_1 BITÁCORA SOBRE MANIPULACIÓN DE DATOS (CRUD).

CRUD DE ESTADIOS EN ROOM

OBJETIVO

Agregar una nueva entidad llamada Stadium a la base de datos Room, manteniendo la misma arquitectura utilizada para Player.

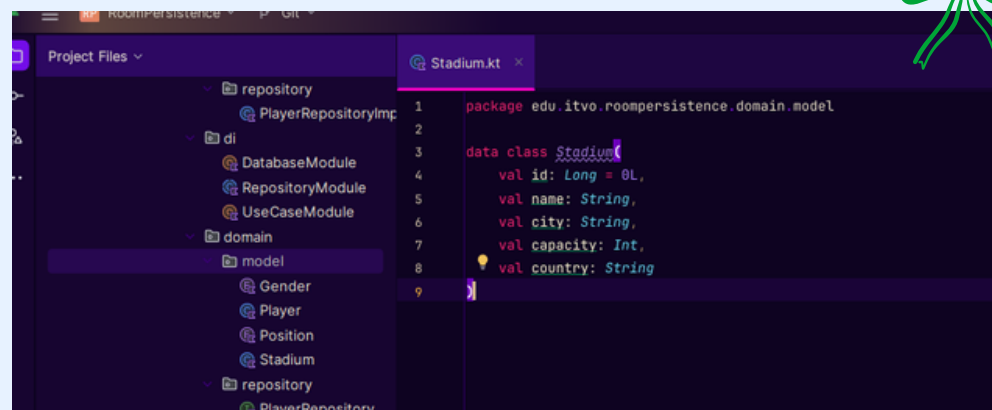
PASO 1. CREAR STADIUM.KT

Se creó la clase Stadium dentro de la capa Domain para representar la información de los estadios.

Resultado

La clase almacena:

- id
- nombre
- ciudad
- capacidad
- país



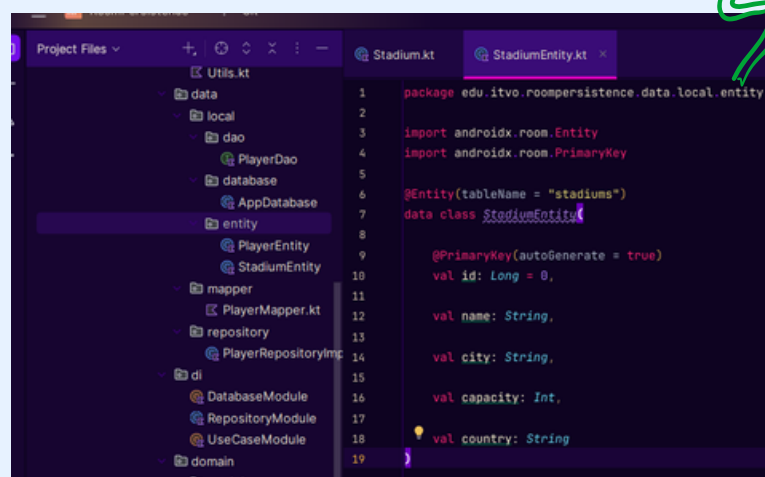
```
1 package edu.itvo.roompersistence.domain.model
2
3 data class Stadium {
4     val id: Long = 0L,
5     val name: String,
6     val city: String,
7     val capacity: Int,
8     val country: String
9 }
```

PASO 2. CREAR STADIUMENTITY.KT

Se implementó una entidad Room para almacenar la información de los estadios dentro de la base de datos local.

Se creó la tabla:

stadiums



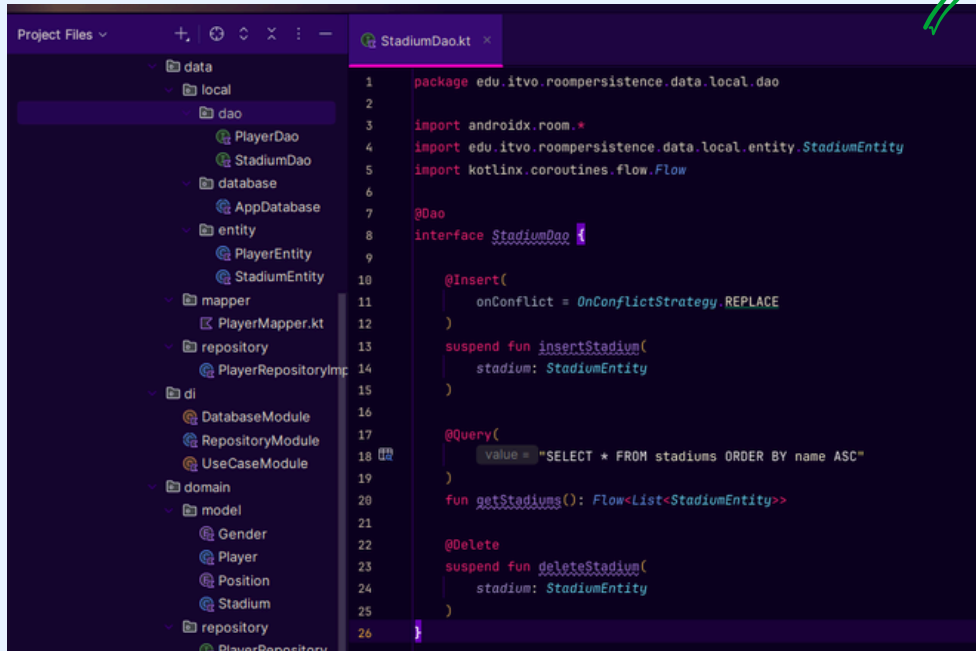
```
1 package edu.itvo.roompersistence.data.local.entity
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "stadiums")
7 data class StadiumEntity {
8
9     @PrimaryKey(autoGenerate = true)
10     val id: Long = 0,
11
12     val name: String,
13     val city: String,
14     val capacity: Int,
15
16     val country: String
17 }
```

PASO 3. CREAR STADIUMDAO.KT

Se desarrolló el DAO para realizar operaciones CRUD sobre la tabla Stadiums.

Operaciones

- Insertar estadio
- Consultar estadios
- Eliminar estadio



```
1 package edu.itvo.roompersistence.data.local.dao
2
3 import androidx.room.*
4 import edu.itvo.roompersistence.data.local.entity.StadiumEntity
5 import kotlinx.coroutines.flow.Flow
6
7 @Dao
8 interface StadiumDao {
9
10     @Insert(
11         onConflict = OnConflictStrategy.REPLACE
12     )
13     suspend fun insertStadium(
14         stadium: StadiumEntity
15     )
16
17     @Query(
18         value = "SELECT * FROM stadiums ORDER BY name ASC"
19     )
20     fun getStadiums(): Flow<List<StadiumEntity>>
21
22     @Delete
23     suspend fun deleteStadium(
24         stadium: StadiumEntity
25     )
26 }
```

PASO 4. MODIFICAR APPDATABASE.KT

Se incorporó la entidad StadiumEntity al esquema de Room y se añadió el acceso mediante StadiumDao.

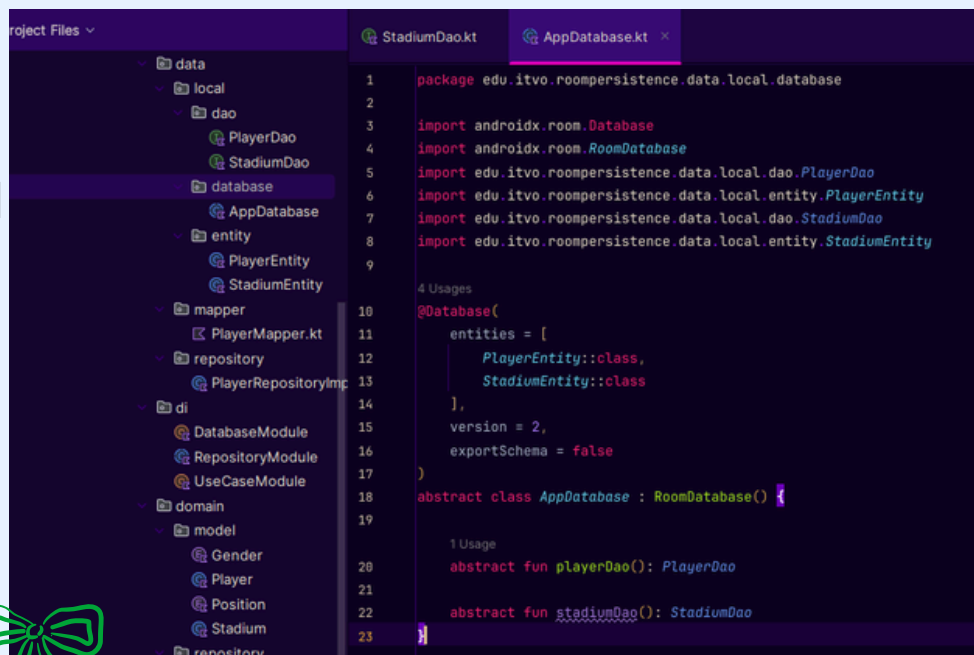
Resultado

La base de datos ahora administra:

- Players
- Stadiums

Antes

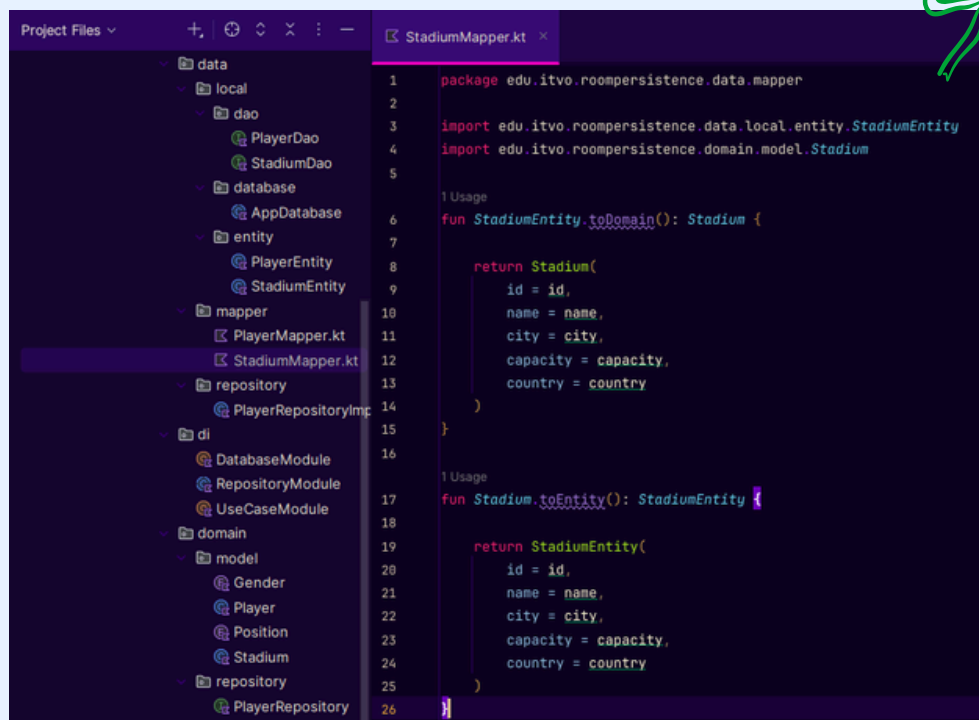
entities = [PlayerEntity::class]



```
1 package edu.itvo.roompersistence.data.local.database
2
3 import androidx.room.Database
4 import androidx.room.RoomDatabase
5 import edu.itvo.roompersistence.data.local.dao.PlayerDao
6 import edu.itvo.roompersistence.data.local.entity.PlayerEntity
7 import edu.itvo.roompersistence.data.local.dao.StadiumDao
8 import edu.itvo.roompersistence.data.local.entity.StadiumEntity
9
10 4 Usages
11 @Database(
12     entities = [
13         PlayerEntity::class,
14         StadiumEntity::class
15     ],
16     version = 2,
17     exportSchema = false
18 )
19 abstract class AppDatabase : RoomDatabase() {
20
21     1 Usage
22     abstract fun playerDao(): PlayerDao
23     abstract fun stadiumDao(): StadiumDao
24 }
```

PASO 5. CREAR STADIUMMAPPER.KT

Se creó la conversión entre la entidad Room y el modelo de dominio para mantener separadas las capas de la aplicación.



```
1 package edu.itvo.roompersistence.data.mapper
2
3 import edu.itvo.roompersistence.data.local.entity.StadiumEntity
4 import edu.itvo.roompersistence.domain.model.Stadium
5
6 1 Usage
7 fun StadiumEntity.toDomain(): Stadium {
8
9     return Stadium(
10         id = id,
11         name = name,
12         city = city,
13         capacity = capacity,
14         country = country
15     )
16
17 1 Usage
18 fun Stadium.toEntity(): StadiumEntity {
19
20     return StadiumEntity(
21         id = id,
22         name = name,
23         city = city,
24         capacity = capacity,
25         country = country
26     )
27 }
```

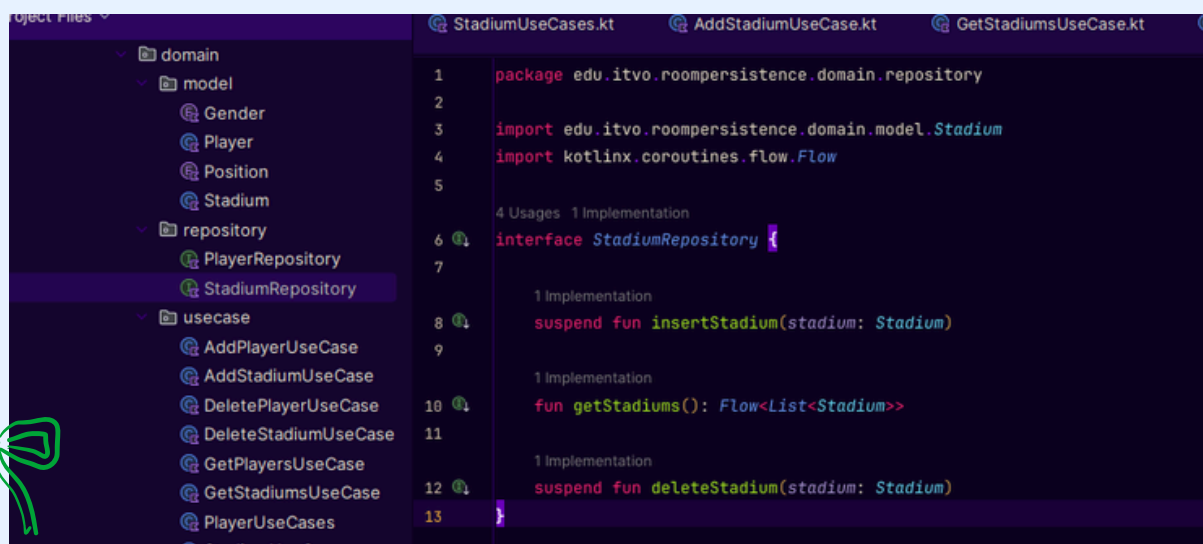
5.1 INTERFAZ STADIUMREPOSITORY

Se implementó la capa de Repository para manejar la comunicación entre la base de datos Room y la lógica de la aplicación.

Se creó una interfaz que define las operaciones básicas del CRUD.

Funciones:

- Insertar estadio
- Obtener lista de estadios
- Eliminar estadio



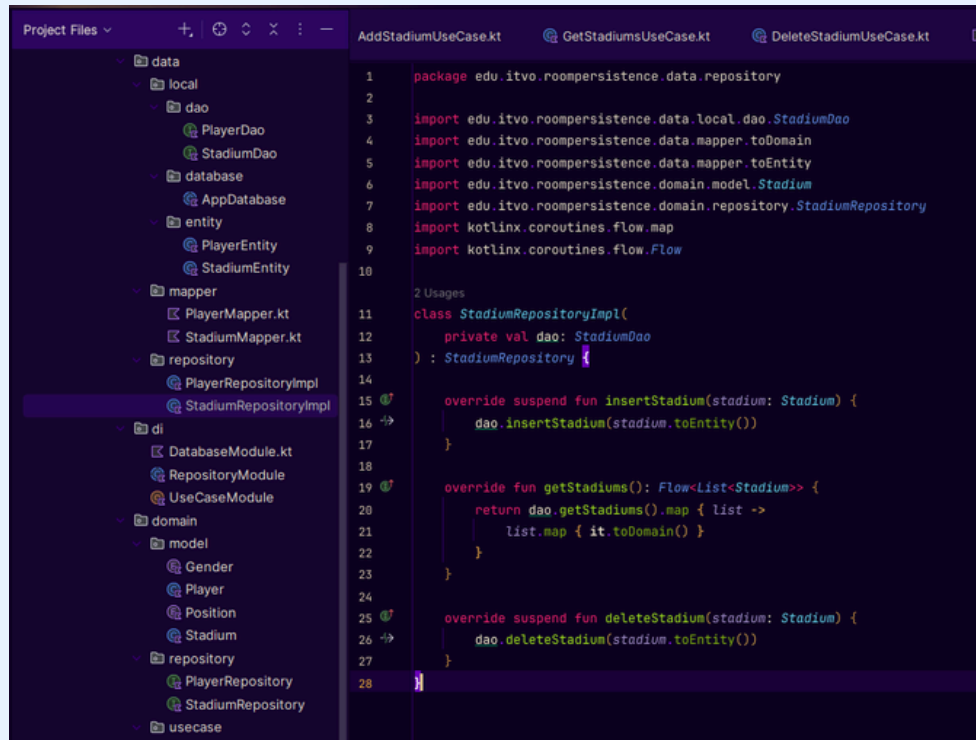
```
1 package edu.itvo.roompersistence.domain.repository
2
3 import edu.itvo.roompersistence.domain.model.Stadium
4 import kotlinx.coroutines.flow.Flow
5
6 4 Usages 1 Implementation
7 interface StadiumRepository {
8
9     1 Implementation
10     suspend fun insertStadium(stadium: Stadium)
11
12     1 Implementation
13     fun getStadiums(): Flow<List<Stadium>>
14
15     1 Implementation
16     suspend fun deleteStadium(stadium: Stadium)
17 }
```

6 IMPLEMENTACIÓN STADIUMREPOSITORYIMPL

Se creó la implementación que conecta la interfaz con Room mediante el DAO.

Función:

- Convierte datos de dominio ↔ entidad
- Ejecuta consultas a la base de datos



```
1 package edu.itvo.roompersistence.data.repository
2
3 import edu.itvo.roompersistence.data.local.dao.StadiumDao
4 import edu.itvo.roompersistence.data.mapper.toDomain
5 import edu.itvo.roompersistence.data.mapper.toEntity
6 import edu.itvo.roompersistence.domain.model.Stadium
7 import edu.itvo.roompersistence.domain.repository.StadiumRepository
8 import kotlinx.coroutines.flow.map
9 import kotlinx.coroutines.flow.Flow
10
11 2 Usages
12 class StadiumRepositoryImpl(
13     private val dao: StadiumDao
14 ) : StadiumRepository {
15
16     override suspend fun insertStadium(stadium: Stadium) {
17         dao.insertStadium(stadium.toEntity())
18     }
19
20     override fun getStadiums(): Flow<List<Stadium>> {
21         return dao.getStadiums().map { list ->
22             list.map { it.toDomain() }
23         }
24     }
25
26     override suspend fun deleteStadium(stadium: Stadium) {
27         dao.deleteStadium(stadium.toEntity())
28     }
29 }
```

RESULTADO

- ✓ Se conectó la lógica de la app con la base de datos
- ✓ Se implementó el patrón Repository
- ✓ Se permitió el manejo de datos sin depender directamente de Room en la UI

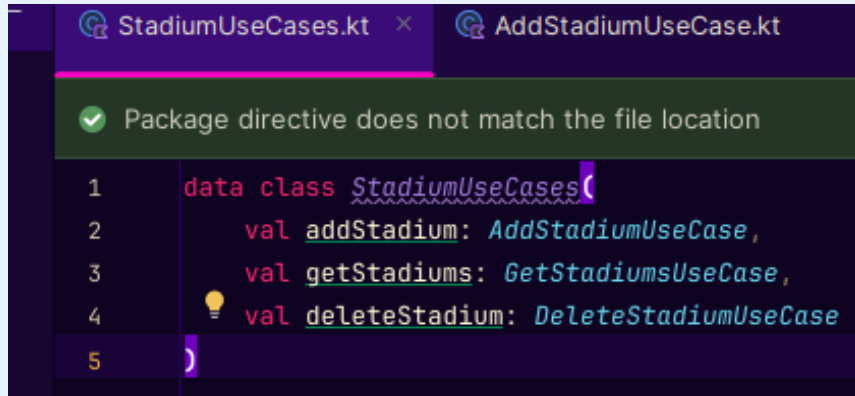
7. CASOS DE USO (USE CASES)

Se crearon los UseCases, que representan las acciones del sistema de forma independiente.

Esto mejora la organización del proyecto (Clean Architecture).

7.1 STADIUMUSECASES

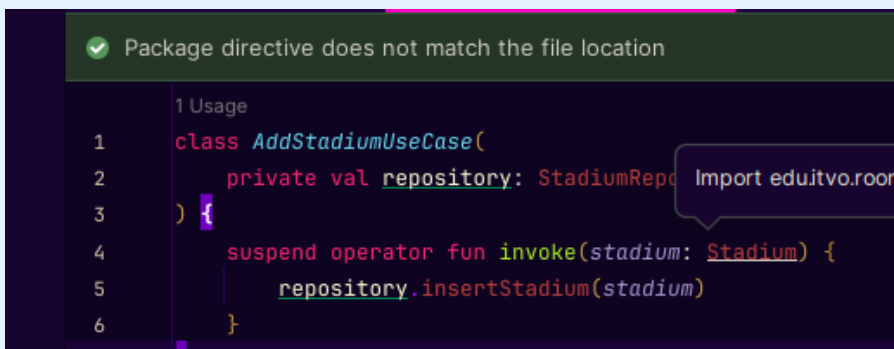
Agrupar todas las operaciones del CRUD en un solo objeto.



The screenshot shows the StadiumUseCases.kt file in an IDE. A green error bar at the top states: "Package directive does not match the file location". The code defines a data class StadiumUseCases with three properties: addStadium, getStadiums, and deleteStadium, each of type AddStadiumUseCase, GetStadiumsUseCase, and DeleteStadiumUseCase respectively.

```
1 data class StadiumUseCases(  
2     val addStadium: AddStadiumUseCase,  
3     val getStadiums: GetStadiumsUseCase,  
4     val deleteStadium: DeleteStadiumUseCase  
5 )
```

7.2 ADDSTADIUMUSECASE



The screenshot shows the AddStadiumUseCase.kt file in an IDE. A green error bar at the top states: "Package directive does not match the file location". The code defines a class AddStadiumUseCase with a private val repository of type StadiumRepository and a suspend operator fun invoke method that calls repository.insertStadium.

```
1 Usage  
1 class AddStadiumUseCase(  
2     private val repository: StadiumRepository  
3 ) {  
4     suspend operator fun invoke(stadium: Stadium) {  
5         repository.insertStadium(stadium)  
6     }  
7 }
```

7.3 GETSTADIUMSUSECASE



The screenshot shows the GetStadiumsUseCase.kt file in an IDE. A green error bar at the top states: "Package directive does not match the file location". The code defines a class GetStadiumsUseCase with a private val repository of type StadiumRepository and an operator fun invoke method that calls repository.getStadiums.

```
1 Usage  
1 class GetStadiumsUseCase(  
2     private val repository: StadiumRepository  
3 ) {  
4     operator fun invoke() = repository.getStadiums()  
5 }
```


8. INYECCIÓN DE DEPENDENCIAS (HILT)

Se configuró Hilt para que las clases se conecten automáticamente sin crear instancias manualmente

8.1 DATABASEMODULE (DAO)

Se agregó el DAO de Stadium a la base de datos.

```
17 object DatabaseModule {
18
19     @Singleton
20     fun provideDatabase(
21         @ApplicationContext context: Context
22     ): AppDatabase {
23         return Room.databaseBuilder(
24             context,
25             klass = AppDatabase::class.java,
26             name = "app_database"
27         )
28         .fallbackToDestructiveMigration()
29         .build()
30     }
31 }
32
33 1 Usage
34 @Provides
35 fun providePlayerDao(db: AppDatabase): PlayerDao = db.playerDao()
36
37 1 Usage
38 @Provides
39 fun provideStadiumDao(db: AppDatabase): StadiumDao = db.stadiumDao()
40 }
```

```
42 return database.stadiumDao()
43 }
```

8.2 REPOSITORYMODULE

Se vinculó la interfaz con su implementación.

```
1 package edu.itvo.roompersistence.di
2
3 import edu.itvo.roompersistence.data.repository.PlayerRepositoryImpl
4 import edu.itvo.roompersistence.data.repository.StadiumRepositoryImpl
5 import edu.itvo.roompersistence.domain.repository.PlayerRepository
6 import edu.itvo.roompersistence.domain.repository.StadiumRepository
7 import dagger.Binds
8 import dagger.Module
9 import dagger.hilt.InstallIn
10 import dagger.hilt.components.SingletonComponent
11 import javax.inject.Singleton
12
13 @Module
14 @InstallIn(value = SingletonComponent::class)
15 abstract class RepositoryModule {
16
17     @Binds
18     @Singleton
19     abstract fun bindPlayerRepository(
20         impl: PlayerRepositoryImpl
21     ): PlayerRepository
22
23     @Binds
24     @Singleton
25     abstract fun bindStadiumRepository(
26         impl: StadiumRepositoryImpl
27     ): StadiumRepository
28 }
```

BOTÓN (AGREGAR ESTADIO)

En tu StadiumScreen

Cuando el usuario presiona el botón:

- o Se leen los textos de los TextFieldname
- o city
- o country
- o capacity

1. Se crea un objeto:

Stadium(...)

TABLA / COLUMNA EN BASE DE DATOS

```
StadiumViewModel.kt
RepositoryModule.kt
UseCaseModule.kt

1 package edu.itvo.roompersistence.data.local.entity
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "stadiums")
7 data class StadiumEntity(
8
9     @PrimaryKey(autoGenerate = true)
10     val id: Long = 0,
11
12     val name: String,
13
14     val city: String,
15
16     val capacity: Int,
17
18     val country: String
19 )
```

```
StadiumRepository.kt
StadiumScreen.kt
PlayerListScreen.kt

17 fun StadiumScreen(
18     ) {
19     label = { Text("Ciudad") }
20 }
21
22 OutlinedTextField(
23     value = country,
24     onChange = { country = it },
25     label = { Text("País") }
26 )
27
28 OutlinedTextField(
29     value = capacity,
30     onChange = { capacity = it },
31     label = { Text("Capacidad") }
32 )
33
34 Spacer(modifier = Modifier.height(12.dp))
35
36 Button(
37     onClick = {
38         if (name.isNotBlank()) {
39             viewModel.addStadium(
40                 Stadium(
41                     name = name,
42                     city = city,
43                     country = country,
44                     capacity = capacity.toIntOrNull() ?: 0
45                 )
46             )
47             name = ""
48             city = ""
49             country = ""
50             capacity = ""
51         }
52     },
53     text = { Text("Agregar estadio") }
54 )
```

BASE DE DATOS (ROOM)

AppDatabase.kt

Room toma el objeto

lo convierte en SQL

lo inserta en SQLite

Ejemplo interno:

INSERT INTO stadiums

(name, city, capacity,

country)

VALUES (...)

```
Project Files
java
edu
itvo
roompersistence
core
utils.kt
data
local
dao
PlayerDao
StadiumDao
database
AppDatabase
entity
PlayerEntity
StadiumEntity
mapper
PlayerMapper.kt
StadiumMapper.kt
repository
PlayerRepositoryImpl
StadiumRepositoryImpl
DatabaseModule
RepositoryModule
UseCaseModule

1 package edu.itvo.roompersistence.data.local.database
2
3 import androidx.room.Database
4 import androidx.room.RoomDatabase
5 import edu.itvo.roompersistence.data.local.dao.PlayerDao
6 import edu.itvo.roompersistence.data.local.dao.StadiumDao
7 import edu.itvo.roompersistence.data.local.entity.PlayerEntity
8 import edu.itvo.roompersistence.data.local.entity.StadiumEntity
9
10 @Database(entities = [
11     PlayerEntity::class,
12     StadiumEntity::class
13 ], version = 1, exportSchema = false)
14 abstract class AppDatabase : RoomDatabase() {
15
16     1 Usage: 1 Implementation
17     abstract fun playerDao(): PlayerDao
18     1 Usage: 1 Implementation
19     abstract fun stadiumDao(): StadiumDao
20 }

Build
Build Output
Build Analyzer

> Task :app:mergeDebugJavaResource
> Task :app:compileDebugJavaWithJavac
> Task :app:transformDebugClassesWithAar
> Task :app:bundleDebugResources
> Task :app:mergeProjectDexDebug
> Task :app:mergeDexDebug
> Task :app:createDebugApkListingFileRedirect
> Task :app:assembleDebug

BUILD SUCCESSFUL in 3m 37s
48 actionable tasks: 13 executed, 27 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.4.1/userguide/configuration_cache_enabling.html
Build Analyzer results available

roompersistence-main > app > src > main > java > edu > itvo > roompersistence > data > local > database > AppDatabase
```


RESULTADO FINAL

Ejecución del sistema CRUD con dos entidades: Players y Stadiums.

Descripción

Se ejecutó la aplicación Android con Room Persistence, integrando dos módulos:

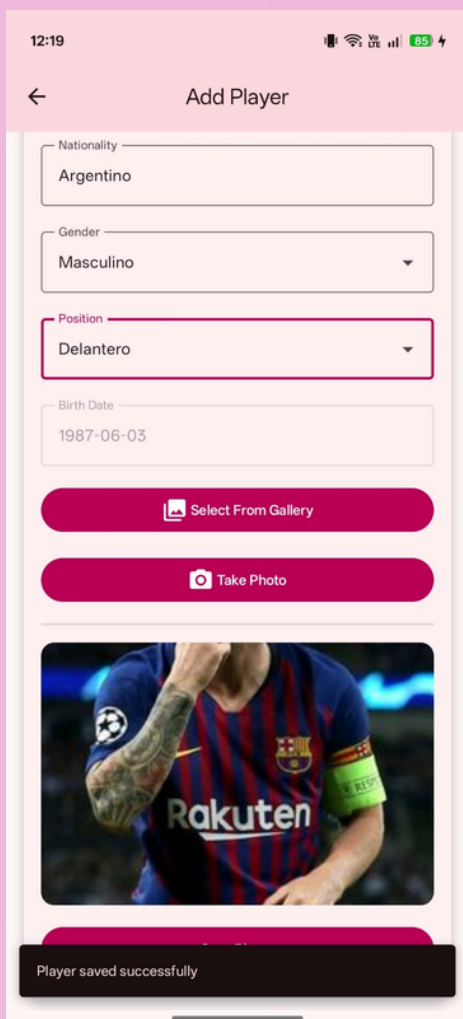
- Gestión de jugadores
- Gestión de estadios

Se validó que ambas entidades funcionan de manera independiente dentro de la misma base de datos local.

Resultado

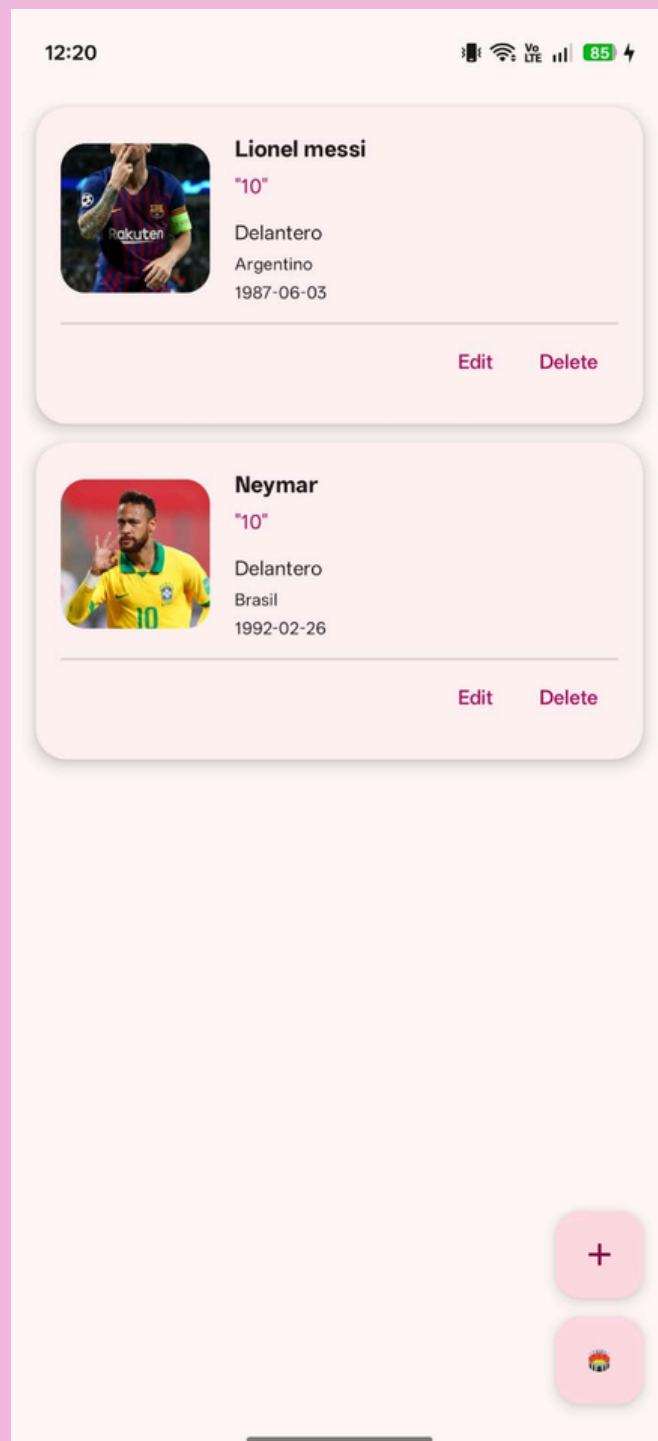
La aplicación permite:

- Registrar jugadores
- Registrar estadios
- Visualizar listas dinámicas
- Eliminar registros



The screenshot shows the 'Add Player' form with the following fields and options:

- Nationality: Argentino
- Gender: Masculino
- Position: Delantero
- Birth Date: 1987-06-03
- Buttons: Select From Gallery, Take Photo
- Image: A photo of Lionel Messi in a Barcelona jersey.
- Message: Player saved successfully



The screenshot shows a list of players with the following details:

Player	Position	Nationality	Birth Date
Lionel messi	Delantero	Argentino	1987-06-03
Neymar	Delantero	Brasil	1992-02-26

Each player entry includes a photo, a name, a position, a nationality, a birth date, and 'Edit' and 'Delete' buttons. A '+' button is visible at the bottom right of the screen.



RESULTADO FINAL

¿Qué se logró?

- ✓ Crear estadios nuevos
- ✓ Listar estadios desde la base de datos
- ✓ Eliminar estadios
- ✓ Guardarlos de forma persistente con Room
- ✓ Separación por capas (Clean Architecture)
- ✓ Inyección de dependencias con Hilt

The screenshot shows the 'Estadios' app interface. At the top, there's a header with the app name and status bar icons. Below the header, there's a form with four input fields: 'Nombre', 'Ciudad', 'País', and 'Capacidad'. A pink 'Agregar estadio' button is positioned below the 'Capacidad' field. Below the form, there's a list of stadiums, each with a flag icon, name, location, and capacity, followed by a pink 'Eliminar' button. The list includes: ESTADIO BBVA (Monterrey, México, 53500), Estadio Azteca (Ciudad de México, México, 87523), and Wembley, Stadium (Londres, Reino unido, 90000).

CONCLUSION

La implementación del módulo de estadios se realizó siguiendo la misma arquitectura del módulo de jugadores, reutilizando el patrón CRUD con Room, Hilt y Clean Architecture.

Esto permitió:

Escalabilidad del sistema

Código organizado por capas

Separación de responsabilidades

Persistencia de datos local en SQLite mediante Room

